

Terraform Core Concepts

1. **Modules** (Reusability & DRY Code)
2. **Dynamic Blocks / Rules** (Reducing Boilerplate)
3. **Data Types** (Variables & Schema Definitions)
4. **Workspaces** (Environment Isolation)
5. **State Management (tfstate)** (Source of Truth & Backend Locking)

1. Terraform Modules

Overview

A **Terraform Module** is a container for multiple resources that are managed together. Modules allow you to write reusable, maintainable, and modular infrastructure code. Instead of repeating configuration across different environments (dev, staging, prod), you wrap common patterns into a module.

Core Structure

A standard module structure looks like this:

```
modules/  
├── aws_s3_bucket/  
│   ├── main.tf  
│   ├── variables.tf  
│   └── outputs.tf
```

Sample Example

Module Definition (modules/aws_s3_bucket/main.tf)

```
variable "bucket_name" {  
  type      = string  
  description = "Name of the S3 bucket"  
}  
  
variable "environment" {  
  type      = string  
  description = "Deployment environment (e.g., dev, prod)"  
}  
  
resource "aws_s3_bucket" "this" {  
  bucket = "${var.bucket_name}-${var.environment}"  
  
  tags = {  
    Environment = var.environment  
  }  
}
```

```

    ManagedBy = "Terraform"
  }
}

output "bucket_arn" {
  value = aws_s3_bucket.this.arn
}

```

Calling the Module (main.tf)

```

module "corporate_storage" {
  source      = "../modules/aws_s3_bucket"
  bucket_name = "acme-corp-data"
  environment = "prod"
}

output "storage_arn" {
  value = module.corporate_storage.bucket_arn
}

```

2. Dynamic Blocks (dynamic)

Overview

In Terraform, some resources accept nested configuration blocks (like ingress rules in Security Groups). Hardcoding these leads to repetitive code. **Dynamic Blocks** allow you to construct nested configuration blocks dynamically based on a complex variable, such as a list or map.

Sample Example: Security Group Ingress Rules

Variables Definition (variables.tf)

```

variable "ingress_rules" {
  type = list(object({
    port      = number
    protocol   = string
    cidr_blocks = list(string)
  }))
  default = [
    { port = 80, protocol = "tcp", cidr_blocks = ["0.0.0.0/0"] },
    { port = 443, protocol = "tcp", cidr_blocks = ["0.0.0.0/0"] },
    { port = 22, protocol = "tcp", cidr_blocks = ["10.0.0.0/8"] }
  ]
}

```

Resource Definition with Dynamic Block (main.tf)

```
resource "aws_security_group" "web_sg" {
  name      = "corporate-web-sg"
  description = "Allow inbound web traffic"

  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port = ingress.value.port
      to_port   = ingress.value.port
      protocol  = ingress.value.protocol
      cidr_blocks = ingress.value.cidr_blocks
    }
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

3. Data Types in Terraform

Overview

Terraform relies on strongly typed variables to enforce data integrity. Types are divided into:

- **Primitive Types:** string, number, bool
- **Complex Types:**
 - `list(<TYPE>)`: Ordered sequence of values.
 - `map(<TYPE>)`: Key-value pairs where all values share the same type.
 - `object({ ... })`: Structural type holding named attributes of differing types.

Sample Example

Variable Definitions (variables.tf)

```
# Primitive Type
variable "app_name" {
  type    = string
  default = "billing-service"
```

```

}

# List Type
variable "availability_zones" {
  type    = list(string)
  default = ["us-east-1a", "us-east-1b", "us-east-1c"]
}

# Map Type
variable "resource_tags" {
  type = map(string)
  default = {
    Owner      = "Finance-IT"
    CostCenter = "CC-1029"
  }
}

# Structural Object Type
variable "db_config" {
  type = object({
    instance_class    = string
    allocated_storage = number
    multi_az          = bool
  })
  default = {
    instance_class    = "db.t3.micro"
    allocated_storage = 20
    multi_az          = false
  }
}

```

Usage Example (main.tf)

```

resource "aws_db_instance" "example" {
  identifier      = var.app_name
  instance_class  = var.db_config.instance_class
  allocated_storage = var.db_config.allocated_storage
  multi_az        = var.db_config.multi_az
  tags            = var.resource_tags
}

```

4. Terraform Workspaces

Overview

Workspaces provide isolated state files within the same working directory. They allow you to reuse the exact same code to deploy parallel environments (e.g., dev, staging, prod) without duplicating .tf files.

- **Default Workspace:** default
- **Current Workspace Helper:** `${terraform.workspace}`

CLI Workflow

1. List existing workspaces

```
terraform workspace list
```

2. Create and switch to a new workspace named 'dev'

```
terraform workspace new dev
```

3. Switch between workspaces

```
terraform workspace select prod
```

4. View current active workspace

```
terraform workspace show
```

Sample Example using `terraform.workspace`

```
locals {  
  # Map instance types according to active workspace  
  instance_type = {  
    dev = "t3.micro"  
    prod = "t3.large"  
  }  
}  
  
resource "aws_instance" "server" {  
  ami      = "ami-0c55b159cbfafa1f0"  
  instance_type = lookup(local.instance_type, terraform.workspace, "t3.micro")  
  
  tags = {  
    Name      = "AppServer-${terraform.workspace}"  
    Environment = terraform.workspace  
  }  
}
```

5. Terraform State (tfstate)

Overview

The terraform.tfstate file maps your real-world cloud resources to your configuration files, tracks metadata, and boosts performance during large-scale deployments.

In enterprise environments:

- **Local State File:** Do **not** commit terraform.tfstate to source control (security & collision risk).
- **Remote State File:** Store state remotely in Amazon S3 or Azure Blob Storage with state locking (via AWS DynamoDB) to prevent concurrent updates.

Sample Remote State Configuration (backend.tf)

```
terraform {  
  required_version = ">= 1.5.0"  
  
  backend "s3" {  
    bucket      = "corporate-tf-state-bucket"  
    key         = "global/s3/terraform.tfstate"  
    region      = "us-east-1"  
    dynamodb_table = "corporate-tf-locks"  
    encrypt     = true  
  }  
}
```

Essential Commands for Managing State

- **Inspect current state:**
terraform state list
- **Show details of a specific resource in state:**
terraform state show aws_instance.server
- **Import existing infrastructure into state:**
terraform import aws_s3_bucket.my_bucket my-existing-bucket-name

Key Takeaways for Corporate Teams

1. **Modules:** Keep infrastructure DRY and promote standard architecture templates across teams.
2. **Dynamic Blocks:** Use sparingly to handle variable-length inner blocks cleanly without duplicate lines.
3. **Data Types:** Leverage explicit type constraint structures (object, map, list) to validate inputs early.

4. **Workspaces:** Great for lightweight single-account environment splitting; consider separate state repos/directories for strict security boundaries between prod and non-prod.
5. **State Management:** Always use a secure remote backend with locking enabled to prevent state corruption across CI/CD pipelines.